

Lecture 19:

Linked List

Linear data structure

• what is linked list -

stores 2 types of information

Data

→ int (1)

→ char (A)

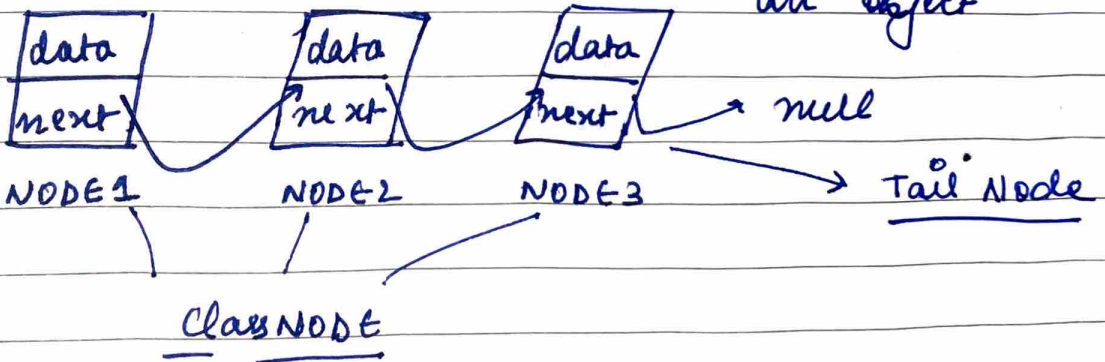
→ boolean (T/F)

→ float (3.45)

Next

→ reference addresses [OOFS]
to the next
possible node

→ A pointer, points to
an object



class Node {

{ int data;

Node next; // Node as type for next as
next is a reference
variable of class

public Node (int data) { // parameter

this.data = data;

this.next = null;

}

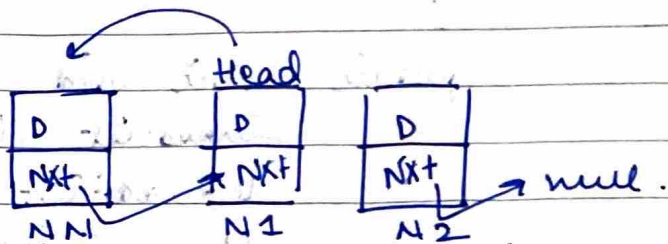
We can traverse the entire linked list using just the head node.

we create methods in linked list to perform operations on linked lists like -

• Add in Linked Lists

(1) add First $\rightarrow O(1)$

- create new node
- new node's next = head node
- head = new Node



```
public class LinkedList {
    public static class Node { // parent class
        int data; // info
        Node next; // sequence variable

        public Node(int data) { // parameter
            this.data = data;
            this.next = null; // empty
        }
    }

    public static Node head;
    public static Node tail;
    // Method
    public void addFirst(int data) {
        // step 1 - create New node
        Node newNode = new Node(data)
    }
}
```


// to check if existing linked list empty then -

if (head == null) {

head = tail = newNode; // head, tail points

return; // to same new

// Step 2 - newNode next points to head

newNode.next = head; // link

// Step 3 - head = newNode

head = newNode;

}

public static void main (String args[]) {

LinkedList ll = new LinkedList();

ll.addFirst(1);

ll.addFirst(2);

}

② Add Last

- Create a node (newNode)

- tail next points to newNode

- tail = newNode.

public void addLast (int data) {

// Step 1: Node newNode = new Node (data);

// check for empty case if (head == null) {

head = tail = newNode;

return;

tail.next = newNode;

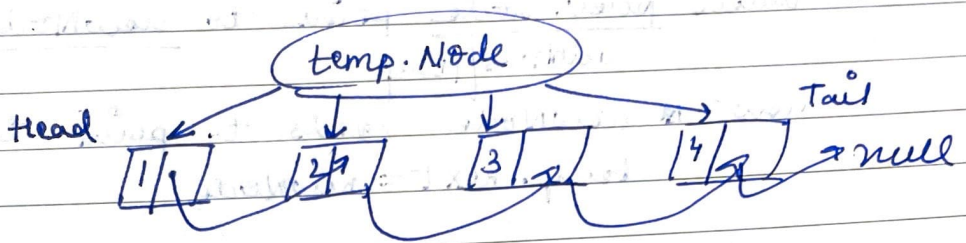
tail = newNode;

// Step 2

// Step 3

③ Print in Linked List

head and tail in linked list are fixed.

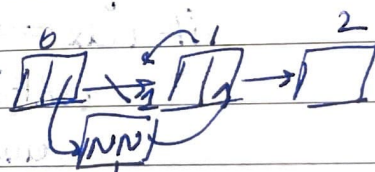


```
public void print() {
    // create a temporary node
    Node temp = head;
    while (temp != null) { // traverse till tail
        // prints data
        System.out.print(temp.data + " ");
        temp = temp.next; // updation
    }
    System.out.println();
}
```

→ Add base case → if linked list is empty
if (head == null) {
 System.out.println("LL is empty");
 return;
}

④ Add in the Middle

add(index, data)



→ temp = 0

- To traverse loop till we reach one previous node to idx
i.e. while (i < idx - 1) {
 temp = temp.next;
 i++;
}

when Node temp = head
i = 0



temp / prev node points to next node in sequence
so break the sequence & store that
address in newNode

- When (temp hits idx-1 condition) i.e. finds previous node then

newNode.next = temp.next;

- When prev. node points to newNode and address/temp

now ~~newNode~~ newNode needs to point to next Node.
temp.next = newNode;

```
public void add(int idx, int data) {
```

```
    Node newNode = new Node(data);
```

```
    Node temp = head;
```

```
    int i = 0;
```

```
    while (i < idx - 1) {
```

```
        temp = temp.next;
```

```
        i++;
```

```
    }
```

```
    // i = idx - 1 is prev; temp → previous node
```

```
    newNode.next =
```

```
    newNode.next = temp.next;
```

```
    temp.next = newNode;
```

```
}
```

→ If someone wishes to add element at index 0
then call addFirst();

→ head controlled → same function

```
if (idx == 0) {
```

```
    addFirst(data);
```

```
    return;
```

⑤ Size of a linked list

create a static size (like tail, head)

```
public static int size;
```

In addFirst()

```
size++;
```

addMiddle()

```
size++;
```

addLast()

```
size++;
```

then in main function $\rightarrow$ `System.out.println(LL.size());`

⑥ RemoveFirst

- Head node delete

- head = head.next $\rightarrow$ shifting head to next node

```
public int removeFirst() {
```

```
    if (size == 0) {
```

```
        System.out.println("LL is empty");
```

```
        return Integer.MIN_VALUE;
```

```
    } else if (size == 1) {
```

```
        int val = head.data;
```

```
        head = tail = null;
```

```
        size = 0;
```

```
        return val;
```

```
    }
```

```
    int val = head.data;
```

```
    head = head.next;
```


7) removeLast()

1. previous next points to null

2. tail = prev [last idx = size - 1
idx - 1 or prev = size - 2

```
public int removeLast() {
    if (size == 0) {
        System.out.println("LL is empty");
        return Integer.MIN_VALUE;
    }
    else if (size == 1) {
        int val = head.data;
        head = tail = null;
        size = 0;
        return val;
    }
    // prev : i = size - 2
    for (int i = 0; i < size - 2; i++) {
        Node prev = head;
        prev = prev.next;
    }
    int val = tail.data;
    prev.next = null;
    tail = prev;
    size--;
    return val;
}
```